

IERG3080 Project Report “TaikoGame”

Group members:

Chan Hei Lun 1155212799

Yau Ngan Wai 1155213082

Cheung Lok Him 1155213640

Submission Date: December 23, 2025

1. Introduction

As a rhythm-action music game, Taiko no Tatsujin needs players strike virtual drums in sync with scrolling notes. In this time project, complete digital version is implemented using WPF and C#. Three features are implemented:

- Baseline: Single-player (song selection, hit judgment, scoring)
- Feature 1: Local 2-player (difficulty, key bindings, speed 1x-4x)
- Feature 2: Online 2-player (SignalR synchronization)

Tech: WPF/C#, ASP.NET Core, three-tier architecture

2. System Structure:

The system is composed of four main layers:

1. Presentation Layer (UI): Including MainWindow , TitleScreen , SongSelectionScreen , GameOverScreen, it handles rendering, animations, and screen transitions.
2. Logic Layer (Game Core): Including MainWindow.xaml.cs, it manages the game loop, timing, scoring, and collision detection.
3. Data Layer (Models): Including NoteData , SongManager , PlayerSettings, it represents static and runtime data.
4. Network: NetworkManager (Facade) ↔ GameHub (room management, broadcasting)

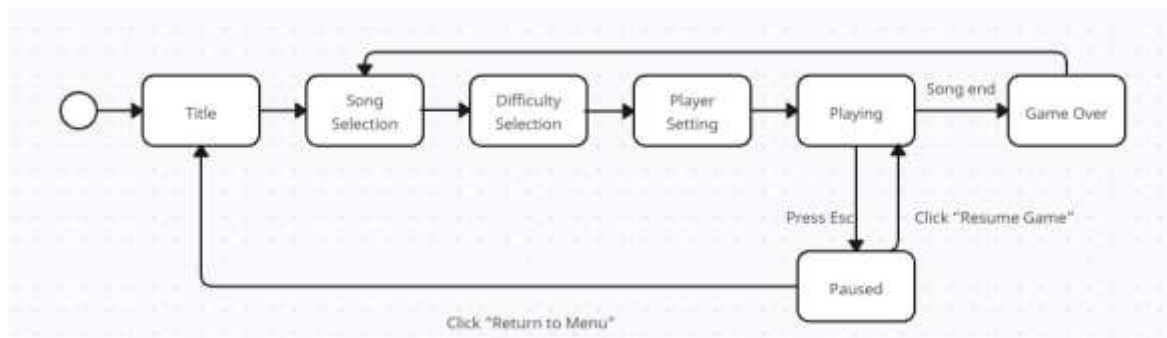
State Machine: Title → Selection → Difficulty → Settings → Gameplay → GameOver

For the online mode, the system includes a client (TaikoGame) and an ASP.NET Core server (TaikoGameServer). The client uses an event-driven NetworkManager for SignalR messaging and a state-driven MainWindow for game flow. The server's GameHub orchestrates rooms and broadcasts.

Class Design

Frontend:

- MainWindow coordinates the entire UI and game flow, modeled with a state machine GameState that transitions through: Title → Song Selection → Difficulty Selection → Player Setting → Game Play → Game Over.



- Visual effect includes particle systems: BackgroundParticleSystem and FloatingImageParticleSystem. Both use timers to spawn and animate elements.
- Reusability: NoteData → Note objects; SongManager shared across screens; identical gameplay logic for local/online

Backend:

- Input handling and hit judgment happen in Window_KeyDown() and CheckHit(), which computes PERFECT/GOOD/MISS, updates scores and combos, and synchronizes to network when online.
- Pause/resume is synchronized across clients via ShowPauseMenu() and ResumePause() and mirrored by incoming network events.
- Songs catalog with metadata is centralized in SongManager.GetSongs(). Class SongManager provides a single source of truth for song metadata, reused across selection and gameplay. NoteData encapsulates note parameters; runtime Note objects reuse color and structure for both players.

NoteData
+Time: double
+IsRed: bool

SongManager
+static GetSongs(): Dictionary

- Online synchronization: The server exposes a SignalR Hub that manages rooms, player assignment, gameplay triggers, and real-time broadcast of events to all

clients in a room. Client-side SignalR wrapper NetworkManager handles connection, event subscription, and command invocation.

- Network integration is initialized and event-wired in InitializeNetwork(). The online entry screen and room lifecycle are handled in ShowOnlineConnectionScreen() , ShowWaitingForPlayerScreen(), and gameplay start handling in the OnStartGameplay() event.

4. Testing Plan

Unit Tests:

- Hit judgment logic: feed synthetic note positions and input to CheckHit() to assert PERFECT/GOOD/MISS thresholds and score/combo changes.
- Note lifecycle: ensure missed notes increment miss count and clear nodes in UpdateNotePositions() and end-of-song branch.
- State transitions: validate screen flow by simulating button/key events and inspecting _currentState .
- Room management: CreateRoom() and JoinRoom() behavior under non-existent, existing, and full-room scenarios.
- Disconnect handling: simulate OnDisconnectedAsync() to assert PlayerLeft signals and room cleanup.

Integration Tests

- Client-server handshake: start server, test whether can connect the client with server
- Real-time synchronization: during play, send judgments from each client and verify mirrored UI changes and score consistency on both ends.
- Test whether the notes can appear from right to left with the music and test the striking of the notes.

Acceptance Tests

- Complete the game: Test the gameplay with 1 Player Mode, 2 Player Mode and Online Mode completely, check whether the client can connect to the host smoothly in online mode.
- Check whether the user interface is easy to read and attractive.

5. Difficulties and solutions

- Difficulty: Keeping both clients' score/combo aggregates aligned under network latency.

Solution: Centralize scoring broadcast in SendScore to include both players' scores and combos, and update both clients uniformly

- Difficulty: Avoid conflicting selections when both clients navigate menus.
Solution: a button "AllowClientControl" toggled by host; client UI shows wait screens until allowed, preventing UI state divergence
- Difficulty: Locating audio files reliably under different working directories.
Solution: Multiple path fallbacks (sounds/, parent sounds/, Assets/) before playing, with debug logging when missing

6. Conclusion

Achievements:

- Three-tier architecture with clean separation
- Five design patterns (Singleton, State Machine, Facade, DTO, Observer)
- Real-time network sync with latency handling
- Comprehensive testing (unit, integration, acceptance)

Future: Extended songs, custom charts, advanced mechanics (yellow bars, balloons), spectator mode, voice chat, accessibility features

List of References

DispatcherTimer:

<https://learn.microsoft.com/zh-cn/dotnet/api/system.windows.threading.dispatchertimer?view=windowsdesktop-10.0>

MediaPlayer:

<https://learn.microsoft.com/zh-cn/dotnet/api/system.windows.media.mediaplayer?view=windowsdesktop-10.0>

SignalR client:

<https://learn.microsoft.com/zh-cn/aspnet/core/signalr/dotnet-client?view=aspnetcore-10.0&tabs=visual-studio>

SignalR hubs:

<https://learn.microsoft.com/zh-cn/aspnet/core/signalr/hubs?view=aspnetcore-10.0>

Real-time ASP.NET with SignalR

<https://dotnet.microsoft.com/en-us/apps/aspnet/signalr>

dotnet publish

<https://learn.microsoft.com/zh-tw/dotnet/core/tools/dotnet-publish>